



Technical Documentation

dm-Verity Mapping Creation (mapping.c)

Team Oriented Project

Group A

Linux Kernel Extension for DM-Verity Authentication, Verification and Setup

Date: **NOVEMBER 2025**

Cooperation with:

**BMW
GROUP**



ROLLS-ROYCE
MOTOR CARS LTD

[BMW Car IT, Ulm](#)

Eva-Helena Zorman

Tobias Kaufmann

Philipp Graf

File Location

../linux/drivers/md/dm-verity-autoboot/mapping.c

1. Introduction

The dm-verity mapping module is a critical kernel component responsible for taking a verified and validated metadata header and using it to instantiate a read-only Device-Mapper (DM) target. This process is essential because the kernel cannot mount the root filesystem until the cryptographic verification layer has been created. The core function, `verity_create_mapping()`, achieves this by building a DM table string and utilizing the kernel's `dm_early_create()` mechanism to establish the device mapping early in the boot sequence.

2. Binary-to-ASCII Encoding

The dm-verity target requires its cryptographic parameters (root hash and salt) to be provided as human-readable, lowercase hexadecimal strings. The module implements a static helper function to perform this necessary conversion.

hex_encode() Function

- Purpose: Converts a raw byte array (`const u8 *src`) into a null-terminated lowercase hexadecimal ASCII string (`char *dst`).
- Mechanism: It iterates through the source buffer, processing each byte in two steps, extracting the upper nibble and the lower nibble to generate two hex characters using a static lookup table (`hexdig`). This is a common technique for efficient binary-to-hex conversion in kernel space.

The function signature and core logic are shown below:

```
26  * @brief Encode a binary buffer into lowercase hexadecimal ASCII string.
27  *
28  *
29  * This function converts each byte of @p src into two ASCII hex digits and stores
30  * the result in @p dst. The resulting string is null-terminated.
31  *
32  * @param src Pointer to source buffer.
33  * @param len Length of the source buffer in bytes.
34  * @param dst Destination buffer (must be at least @c len*2+1 bytes).
35  */
36  static void hex_encode(const u8 *src, size_t len, char *dst)
37  {
38      static const char hexdig[] = "0123456789abcdef";
39      size_t i;
40
41      for (i = 0; i < len; i++) {
42          dst[2 * i] = hexdig[(src[i] >> 4) & 0xf];
43          dst[2 * i + 1] = hexdig[src[i] & 0xf];
44      }
45      dst[2 * len] = '\0';
46  }
```

3. Device-Mapper Table Construction and Installation

The `verity_create_mapping()` function is the entry point for DM instantiation. It handles endianness conversion, validation, string formatting, and the final call to the DM subsystem.

3.1. Metadata Extraction and Deep Validation

The function begins by extracting fields from the verified metadata header (`struct verity_metadata_header *h`) using the correct little-endian-to-CPU (`leX_to_cpu`) conversions to handle cross-architecture compatibility.

Following extraction, the code enforces several critical security and integrity checks before proceeding:

- Hash Algorithm Check: Only sha256 is supported, failing immediately with `-EINVAL` otherwise.
- Block Size Consistency: It strictly mandates that the data block size and hash block size must be equal (`data_bs != hash_bs`).
- Sector Alignment: The data block size is checked to ensure it is a multiple of 512 bytes (`data_bs < 512 || (data_bs & 511)`), preventing misaligned I/O operations.

The validation code is:

```
102     if (strcmp(algo, "sha256") != 0) {
103         pr_err("%s: only sha256 supported (algo=%s)\n", DM_MSG_PREFIX, algo);
104         return -EINVAL;
105     }
106
107     if (!data_blocks || !data_bs || !hash_bs)
108         return -EINVAL;
109
110     if (data_bs != hash_bs)
111         return -EINVAL;
112
113     if (data_bs < 512 || (data_bs & 511)) {
114         pr_err("%s: data_block_size must be multiple of 512\n", DM_MSG_PREFIX);
115         return -EINVAL;
116     }
```

3.2. Parameter String Formatting

The validated binary fields are converted and formatted into the dm-verity table string. This string serves as the kernel's instructions for creating the mapped device. The string is dynamically allocated using `kasprintf` (`GFP_KERNEL, ...`) and follows a precise order:

```
128     params = kasprintf(GFP_KERNEL,
129         "%u %u:%u %u:%u %u %u %llu %llu %s %s %s",
130         version,
131         MAJOR(data_dev), MINOR(data_dev),
132         MAJOR(data_dev), MINOR(data_dev),
133         data_bs, hash_bs,
134         (unsigned long long)data_blocks,
135         (unsigned long long)hash_start_block,
136         algo, root_hex, salt_hex);
137     if (!params)
138         return -ENOMEM;
```

The parameters encoded are: verity version, device IDs (data and hash), block sizes, data block count, hash start block, hash algorithm, hex-encoded root hash, and hex-encoded salt.

3.3. Device-Mapper Invocation (dm_early_create)

The function prepares the Device-Mapper control structures (`dm_ioct1` and `dm_target_spec`) before calling the creation function.

- The `dm_ioct1` structure sets the target name to `verity_root` and sets the flag to `DM_READONLY_FLAG` (`dmi.flags = DM_READONLY_FLAG`), ensuring the device cannot be written to.
- The `dm_target_spec` defines the mapped area, starting at sector 0 and covering the entire verified data region (`spec->length = (u64)num_data_sectors`).

The final, crucial step is the call to `dm_early_create()`:

```
166     ret = dm_early_create(&dmi, spec_array, params_array);
167     kfree(spec);
168     kfree(params);
169
170     if (ret) {
171         pr_err("%s: dm_early_create() failed: %d\n", DM_MSG_PREFIX, ret);
172         return ret;
173     }
```

The function `dm_early_create()` attempts to instantiate the DM device. If successful (returns 0), the new device, typically `/dev/dm-0`, is created under the name `verity_root` and is immediately ready to be mounted by the kernel as the authenticated root filesystem.

End of Document